

Ex-no: 11	FIFO AND LRU PAGE REPLACEMENT ALGORITHM
30/04/2026	

Aim

To implement and compare the **FIFO (First In First Out)** and **LRU (Least Recently Used)** Page Replacement Algorithms used in Operating Systems for memory management.

Algorithm

1. Start the program.
2. Input the number of pages, reference string, and number of frames.
3. Apply **FIFO Page Replacement Algorithm**:
 - Insert pages into frames in the order they arrive.
 - When a page fault occurs and frames are full, replace the oldest page in memory.
 - Count the number of page faults.
4. Apply **LRU Page Replacement Algorithm**:
 - Insert pages into frames based on recent usage.
 - When a page fault occurs and frames are full, replace the page that has not been used for the longest time.
 - Count the number of page faults.
5. Display the frame status and total page faults for both algorithms.
6. Stop the program.

CODE:

FIFO.C:

```
//FIFO
```

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, frames;
```

```
// =====
```

```
// INPUT NUMBER OF PAGES
```

```
// =====
```

```
printf("Enter number of pages in reference string: \n");
```

```
scanf("%d", &n);
```

```
/*
```

Example:

n = 7 pages

Reference String:

1 3 0 3 5 6 3

```
*/
```

```
// =====
```

```
// DECLARE REFERENCE STRING ARRAY
```

```
// =====
```

```
int ref[n];
```

```
// =====
```

```
// INPUT REFERENCE STRING
```

```
// =====
```

```
printf("Enter the reference string: \n");
```

```
for(int i = 0; i < n; i++) {
```

```
    scanf("%d", &ref[i]);
```

```
}
```

```
// =====
```

```
// INPUT NUMBER OF FRAMES
```

```
// =====
```

```
printf("Enter number of frames: \n");
```

```
scanf("%d", &frames);
```

```
/*
```

Frames = Number of memory slots available

Example:

```
frames = 3
```

```
*/
```

```
// =====
```

```
// DECLARE FRAME ARRAY
```

```
// =====
```

```

int frame[frames];

// Initialize frames as empty (-1)
for(int i = 0; i < frames; i++) {
    frame[i] = -1;
}

// =====
// FIFO POINTERS AND COUNTERS
// =====

int front = 0; // points to the oldest page in frames
int faults = 0; // counts page faults
int hits = 0; // counts page hits

// =====
// FIFO PAGE REPLACEMENT LOOP
// =====

for(int i = 0; i < n; i++) {

    int found = 0; // flag to check if page is already in frame

    // =====

```

```

// CHECK FOR PAGE HIT

// =====

for(int j = 0; j < frames; j++) {

    if(frame[j] == ref[i]) {

        /*
        Page is already in memory
        → HIT
        */

        found = 1;

        hits++;

        break;

    }

}

// =====

// PAGE FAULT HANDLING

// =====

if(!found) {

    /*

```

Page is not in memory → FAULT

FIFO Replacement:

- Replace the oldest page (pointed by `front`)

- Move `front` to next frame cyclically

*/

```
frame[front] = ref[i];
```

```
front = (front + 1) % frames;
```

```
faults++;
```

```
}
```

```
}
```

```
// =====
```

```
// FINAL OUTPUT
```

```
// =====
```

```
printf("\nTotal Page Faults: %d\n", faults);
```

```
printf("Total Page Hits: %d\n", hits);
```

```
return 0;
```

```
}
```

LRU.C:

```
//LRU
```

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, frames;
```

```
    // =====
```

```
    // INPUT NUMBER OF PAGES
```

```
    // =====
```

```
    printf("Enter number of pages in reference string: \n");
```

```
    scanf("%d", &n);
```

```
    /*
```

```
    Example:
```

```
    n = 5 pages
```

```
    Reference String:
```

```
    1 2 3 4 5
```

```
    */
```

```
// =====  
// DECLARE REFERENCE STRING ARRAY  
// =====
```

```
int ref[n];
```

```
// =====  
// INPUT REFERENCE STRING  
// =====
```

```
printf("Enter the reference string: \n");
```

```
for(int i = 0; i < n; i++) {  
    scanf("%d", &ref[i]);  
}
```

```
// =====  
// INPUT NUMBER OF FRAMES  
// =====
```

```
printf("Enter number of frames: \n");  
scanf("%d", &frames);
```

```
/*
```

```
frames = number of memory slots available
```


Example: frames = 3

```
*/
```

```
// =====
```

```
// DECLARE FRAME AND TIME ARRAYS
```

```
// =====
```

```
int frame[frames]; // stores pages in memory
```

```
int time[frames]; // stores "last used time" for each frame
```

```
// =====
```

```
// INITIALIZATION
```

```
// =====
```

```
for(int i = 0; i < frames; i++) {
```

```
    frame[i] = -1; // empty frame
```

```
    time[i] = 0; // time = 0 initially
```

```
}
```

```
int faults = 0; // counts page faults
```

```
int hits = 0; // counts page hits
```

```
int counter = 0; // global counter to track page usage
```

```
// =====
```

```
// LRU PAGE REPLACEMENT LOOP
```

```
// =====
```

```
for(int i = 0; i < n; i++) {
```

```
    int found = 0; // flag to check if page is already in memory
```

```
    // =====
```

```
    // CHECK FOR PAGE HIT
```

```
    // =====
```

```
    for(int j = 0; j < frames; j++) {
```

```
        if(frame[j] == ref[i]) {
```

```
            /*
```

```
            Page is already in memory → HIT
```

```
            Update time of this frame to current counter
```

```
            */
```

```
            hits++;
```

```
            counter++;
```

```
            time[j] = counter; // most recently used
```

```
            found = 1;
```

```
            break;
```

```
        }
```

```
    }
```

```
// =====
```

```
// PAGE FAULT HANDLING
```

```
// =====
```

```
if(!found) {
```

```
    /*
```

```
    Page is not in memory → FAULT
```

```
    Find **Least Recently Used (LRU)** frame:
```

```
    - Frame with smallest `time[]` value
```

```
    - Replace that frame with new page
```

```
    */
```

```
    int lruIndex = 0; // assume frame 0 is LRU initially
```

```
    for(int j = 1; j < frames; j++) {
```

```
        if(time[j] < time[lruIndex]) {
```

```
            lruIndex = j;
```

```
        }
```

```
    }
```

```
    counter++;          // increment global time
```

```
    frame[lruIndex] = ref[i]; // load new page
```

```

        time[lruIndex] = counter; // update its usage time

        faults++;
    }

}

// =====
// FINAL OUTPUT
// =====

printf("\nTotal Page Faults: %d\n", faults);
printf("Total Page Hits: %d\n", hits);

return 0;
}

```

OUTPUT:

Enter number of pages in reference string: 7

Enter the reference string:

1 3 0 3 5 6 3

Enter number of frames: 3

1. FIFO Page Replacement

Total Page Faults: 6

Total Page Hits: 1

2. LRU Page Replacement

Total Page Faults: 5

Total Page Hits: 2

Result

Thus, the **FIFO** and **LRU Page Replacement Algorithms** were implemented successfully, and the page replacement process along with page faults was analyzed for both algorithms.